



Program: BS Computer Science

Semester: Fall 2025

Credit Hour(s): 1

Pre-requisite(s): EE1005 Digital Logic Design

Instructor: Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

Lab # 4: Register Indirect Addressing, Offset Addressing, Conditional Jumps

1. Lab Objectives

- Understand the concept of addressing modes in assembly language.
- Learn how the CPU accesses data using **Register Indirect Addressing** and **Register + Offset Addressing**.
- Explore **segment association** and how memory segmentation works in x86 architecture.
- Understand **address wraparound** and its effect on memory addressing.

2. Addressing Modes Covered

2.1 Difference between Direct and Indirect Addressing

- Direct Addressing → “Go directly to that address.”
- Indirect Addressing → “Register points to the address, then go there.”

Direct Addressing	Indirect Addressing
Instruction itself (label/constant) holds the address	Register holds the address
Address is fixed, always accesses the same location	Address is flexible, can access different locations
Used for simple access to known variables	Useful for arrays, loops, pointers
Example: <code>mov ax, [data]</code>	Example: <code>mov bx, data</code> <code>mov ax, [bx]</code>

2.2 Register Indirect Addressing

In register indirect addressing, a **register holds the memory address** of the data instead of the data itself.

Example:

```
mov bx, 2000h      ; BX contains memory address
mov ax, [bx]       ; AX loads the value stored at memory location DS:2000h
```

Here, `[bx]` means “go to the memory address stored in BX and fetch the data”



The register acts as a *pointer to a memory location*. The assembler knows to interpret the brackets [] as an instruction to go to the memory address held in the register, not the value of the register itself.

This is useful for **arrays, strings or pointers**, as we can change the register to point to different locations dynamically.

2.3 Register + Offset Addressing

This mode is similar to register indirect addressing, but it adds a constant value (an offset) to the address stored in the register.

Here, the **effective address** is calculated as: $Effective\ Address = Base\ Register + Offset$

Offset can be a constant value or another register

Example:

```
mov bx, 2000h           ; Base address
mov ax, [bx+4]          ; AX loads data from DS:2004h
```

This allows accessing elements of **arrays or structures** in memory.

2.4 Segment Association

In x86 architecture, memory is divided into segments:

- **CS** (Code Segment) → For program instructions
- **DS** (Data Segment) → For variables/data
- **SS** (Stack Segment) → For stack
- **ES** (Extra Segment) → For additional data

By default:

- **BP** and **SP** use **SS** (Stack Segment)
- **SI**, **DI**, and **BX** use **DS** (Data Segment)

Example:

```
mov ax, [bx]            ; Accesses memory at DS:BX
mov ax, [es:bx]          ; Accesses memory at ES:BX
```

Segments provide a way to access different parts of memory. Segment association ensures correct memory reference of the addresses upto 1MB (20-bit address) with 16 bit registers. The segment register holds the starting address of a 64KB block, and the effective address is an offset from that starting address. This allows for addressing more than 64KB of memory, which became as a physical address.



2.5 Address Wraparound

When memory offset exceeds **FFFFh (65535)**, it wraps around to the start of the segment.

Example: If BX holds **FFFDh** and you add 5, the new address is not **10002h**. Instead, it wraps around.

$$\text{FFFDh} + 5\text{h} = 10002\text{h}$$

Since addresses are 16-bit, the 1 is discarded, and the effective address becomes **0002h**

This happens because offset is only **16 bits**, so it overflows after **FFFFh**. To handle this, we use pointers or manually adjust segment.

3. Examples

Example 1:

```
[org 0x0100]
    mov bx, myArray          ; Indirect Addressing
    mov ax, [bx]
    add bx, 2
    mov cx, [bx]
mov  ax, 0x4c00
int  0x21
myArray dw 10, 20, 30
```

Example 2:

```
[org 0x0100]
    xor ax, ax
    xor bx, bx

    mov ah, [data]
    mov bl, [data+1]
    add ah, bh

    mov bh, [data+2]
    add ah, bh

    mov [data+3], ah
mov  ax, 0x4c00
int  0x21

data: db 10, 20, 30, 0
```

Example 3:

```
[org 0x0100]
    xor ax, ax
    xor cx, cx
```



```
mov  al, [data]

mov  bl, [data+1]
add  al, bl

mov  bl, [data+2]
add  al, bl

mov  [data+3], al
mov  ax, 0x4c00
int  0x21

data: db  10, 20, 30, 0
```

Example 4:

```
[org 0x0100]
xor  ax, ax

mov  bx, data

add  ax, [bx]
add  bx, 2

add  ax, [bx]
add  bx, 2

add  ax, [bx]
; add bx, 2

mov  [result], ax
mov  ax, 0x4c00
int  0x21

data:  dw  10, 20, 30
result: dw  0
```

Example 5:

```
[org 0x0100]
mov  ax, 0
mov  cx, 3           ;loop counter = 3
mov  bx, data

loop_start:
add  ax, [bx]        ;add current word to AX
add  bx, 2           ;move to next word
```



```
    dec    cx                ;decrement counter
    jnz    loop_start        ;repeat until CX = 0

mov    [result], ax

mov    ax, 0x4c00
int    0x21

data:   dw    10, 20, 30
result: dw    0
```

4. Lab Tasks

Task 1:

- Store values at DS:1200h and DS:1202h.
- Use BX as a pointer to load these values into registers AX and CX.

Task 2:

- Take three numbers and store in memory.
- Use a register as an accumulator.
- Access values from memory indirectly using an offset in the instruction.
- Add them together and save the result in the last memory location.

Task 3:

Take three numbers which are stored consecutively in memory (data: db 7, 12, 20, 0)

Write a program to:

- Load each number one by one into registers.
- Add them together.
- Store the result back into the 4th memory location.

Task 4:

- Store three word values in memory (dw 7, 12, 20)
- Use a base register as a pointer to access them one by one.
- Add them into **AX**.
- Store the result into a separate memory variable.

Task 5:

- Store five word values in memory (dw 7, 12, 20, 35, 9)
- Use **CX** as a counter to repeat the addition process for each value.
- Use **SI** as a pointer to access memory sequentially.
- Store the total sum into the variable **result**.